ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ
КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Алгоритми на графах

Звіт про виконання лабораторної роботи № 7

Виконав(ла): ст. 2 курсу, групи 2ПІ-25Б
Коваль Олександр

Перевірив: викладач Карась О.В.

Варіант 12

Вага мінімального кістяка неорієнтованого зваженого зв'язкового графа

МЕТА РОБОТИ

Ознайомитися з алгоритмами на графах; реалізувати на мові C++ програму для знаходження ваги мінімального кістяка (MST) неорієнтованого зваженого зв'язкового графа; оцінити ефективність реалізованого алгоритму.

ЗАВДАННЯ

Варіант 12. Від вас потрібно визначити вагу мінімального кістяка для неорієнтованого зваженого зв'язкового графа.

- Формат вхідних даних: у першому рядку — N і M ($1 \le N \le 100$; $1 \le M \le 6000$), де N — кількість вершин, M — кількість ребер. У кожному з наступних M рядків — трійка A, B, C: номери вершин та вага ребра (натуральне число ≤ 30000).
- Формат вихідних даних: одне число — шукана вага мінімального кістяка.

Приклад:

Вхідні дані: 3 3 / 1 2 1 / 2 3 2 / 3 1 3

Вихідні дані: 3

ОПИС АЛГОРИТМУ

Для розв'язку задачі застосовано алгоритм Краскала. Ребра графа сортуються за вагою у порядку зростання; далі за допомогою системи неперетинних множин (DSU / Union-Find) ребра додаються до кістяка, якщо їх кінці належать різним компонентам зв'язності. Процес триває, доки не буде додано N-1 ребер.

- Складність сортування ребер: O(M log M).
- Операції DSU (find/unite) зі стисненням шляху та об'єднанням за рангом: майже $O(\alpha(N))$ на операцію, де α — обернена функція Аккермана.
- Загальна складність: $O(M \log M + M \cdot \alpha(N))$, що ефективно для $M \le 6000$.

КОД ПРОГРАМИ

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
```

```
using namespace std;

// ============================================================
// Лабораторна робота №7 — варіант 12.
// Задача: вага мінімального кістяка (MST) неорієнтованого
// зваженого зв'язкового графа.
// Алгоритм: Краскала (сортування ребер + DSU).
// Вхід: input.txt, вихід: output.txt.
// ============================================================

// Ребро графа: дві вершини та вага.
struct Edge {
    int u, v, w;
};

// Система неперетинних множин (DSU / Union-Find) для перевірки
// зв'язності компонент при побудові кістяка.
struct DSU {
    vector<int> parent, rnk;
    DSU(int n) : parent(n + 1), rnk(n + 1, 0) {
        for (int i = 0; i <= n; i++) parent[i] = i;
    }
    // Пошук представника множини зі стисненням шляху.
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    // Об'єднання двох множин; true якщо вони були різними.
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (rnk[a] < rnk[b]) swap(a, b);
        parent[b] = a;
        if (rnk[a] == rnk[b]) rnk[a]++;
        return true;
    }
};

int main() {
    ifstream in("input.txt");
    if (!in) { cerr << "Не вдалося відкрити input.txt\n"; return 1; }

    int n, m;
    if (!(in >> n >> m)) { cerr << "Помилка читання N, M\n"; return 1; }

    vector<Edge> edges;
    edges.reserve(m);
    for (int i = 0; i < m; i++) {
        Edge e;
        if (!(in >> e.u >> e.v >> e.w)) { cerr << "Помилка читання ребра\n"; return 1;
}
        edges.push_back(e);
    }
    in.close();

    // Краскал: сортуємо ребра за вагою (зростання).
```

```
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.w < b.w;
    });

    DSU dsu(n);
    long long total = 0;   // вага мінімального кістяка
    int used = 0;          // кількість доданих ребер (має стати n-1)
    for (const auto& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            total += e.w;
            if (++used == n - 1) break; // кістяк побудовано
        }
    }

    ofstream out("output.txt");
    if (!out) { cerr << "Не вдалося створити output.txt\n"; return 1; }
    out << total << "\n";
    out.close();

    cout << "Вага мінімального кістяка: " << total << "\n";
    return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ

Програма скомпільована командою: g++ -O2 -std=c++20 main.cpp -o main.exe

Тест 1 (приклад із завдання):

Вхідний файл input.txt:

3 3
1 2 1
2 3 2
3 1 3

Вихідний файл output.txt: 3

Результат збігається з очікуваним значенням (3).

БЛОК-СХЕМА АЛГОРИТМУ

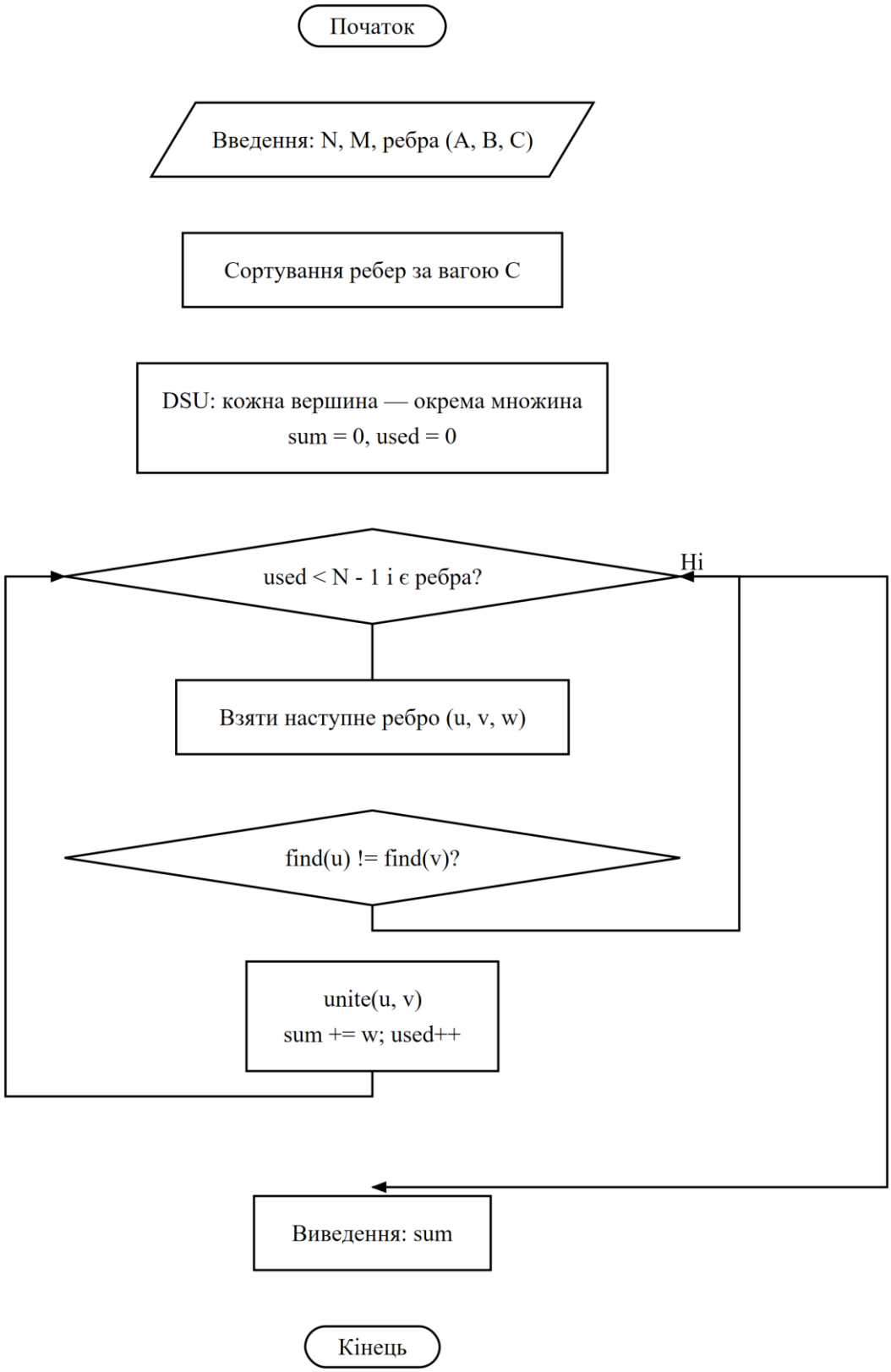

Початок
Введення: N, M, ребра (A, B, C)
Сортування ребер за вагою C
DSU: кожна вершина — окрема множина
sum = 0, used = 0
used < N - 1 і є ребра?
Ні
Взяти наступне ребро (u, v, w)
find(u) != find(v)?
unite(u, v)
sum += w; used++
Виведення: sum
Кінець

Рисунок 1 — Блок-схема алгоритму Краскала (ГОСТ 19.701-90)

UML-ДІАГРАМИ ПРОГРАМНОЇ СИСТЕМИ

На рисунках наведено UML-діаграми програмної системи лабораторної роботи №7: діаграму класів, діаграму послідовності та діаграму станів.

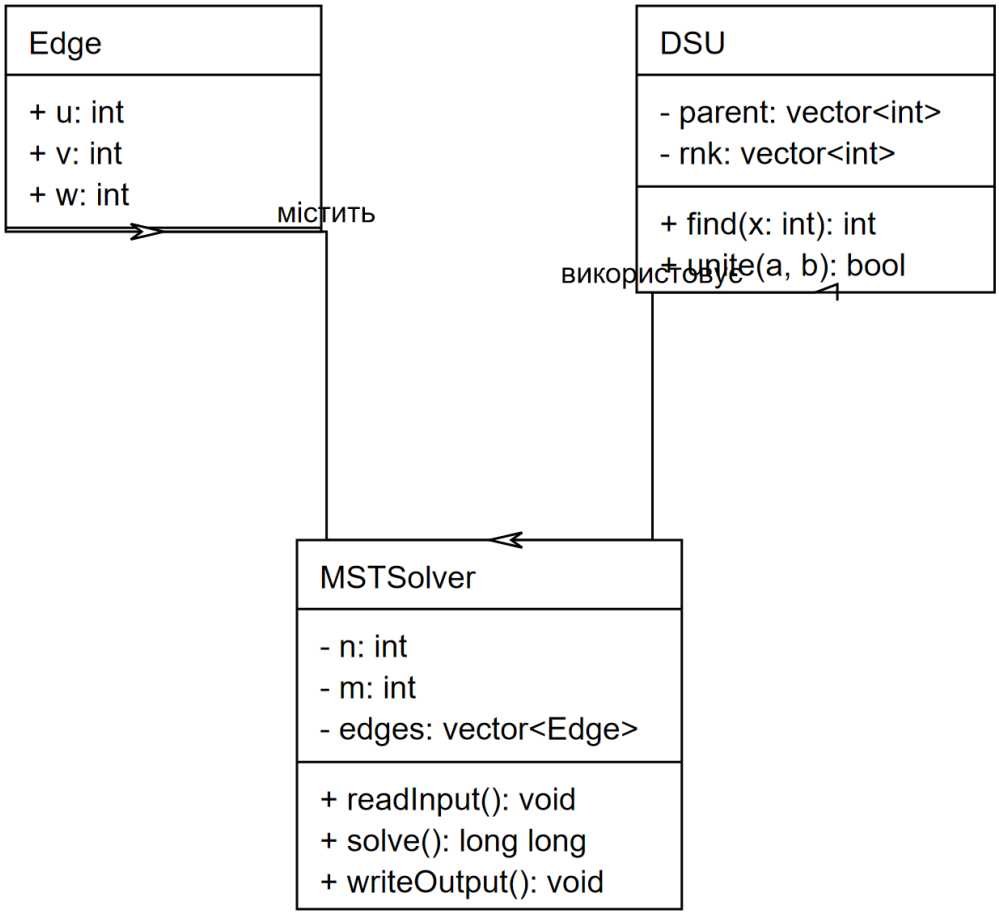


Рисунок 2 — Діаграма класів

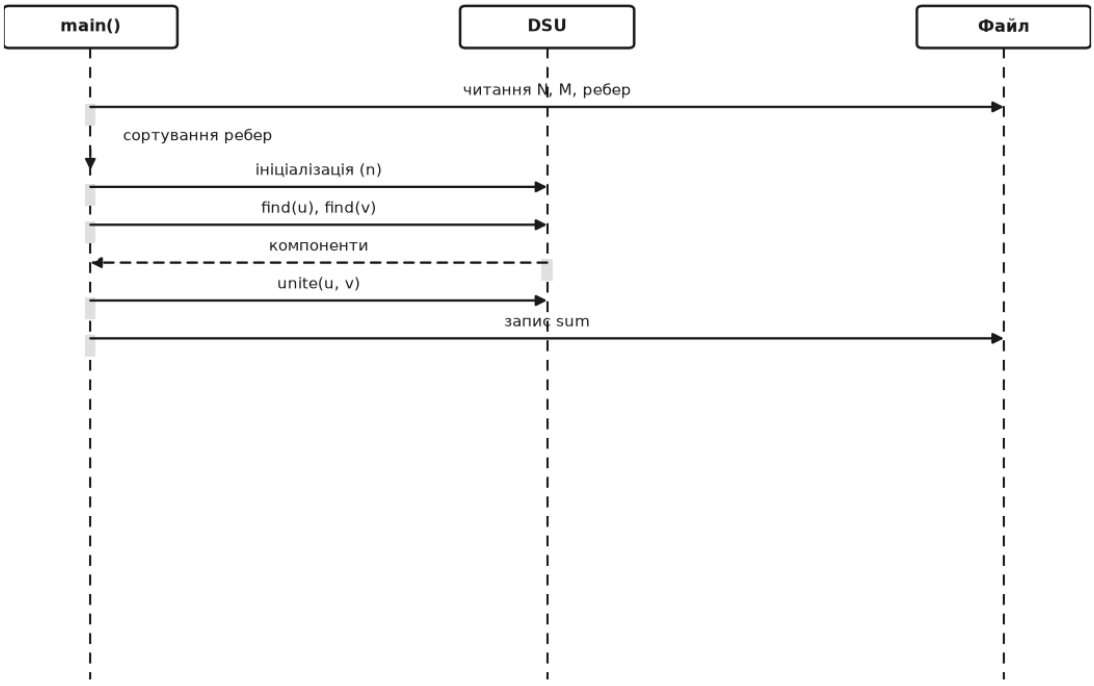


Рисунок 3 — Діаграма послідовності

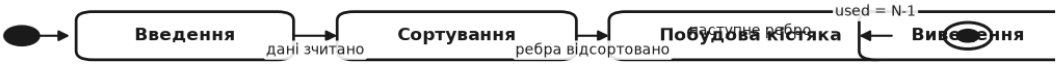


Рисунок 4 — Діаграма станів

ВИСНОВКИ

Реалізовано програму для знаходження ваги мінімального кістяка неорієнтованого зваженого зв'язкового графа за алгоритмом Краскала з використанням DSU. Алгоритм має складність $O(M \log M)$, що є ефективним для заданих обмежень ($N \leq 100$, $M \leq 6000$). Правильність роботи підтверджено тестом на прикладі з умови завдання.